CNG 351 - Assignment 1

Data Management and File Structures

Fall 2026-2027

Entity-Relationship Database Designs

Team / Group ID: 8

Team member	**Student ID**
Osman Caner Cakmak	2584829
Bedri Can Guneser	2585032

Notation and model organization

The diagrams use eight views: 1A-1B for Use Case 1, 2A-2C for Use Case 2 and 3A-3C for Use Case 3. When an entity appears in more than one view, it is the same entity, with the same key, attributes and relationships.

Chen notation

Symbol / marking	Meaning
Single rectangle	An entity with its own key.
Double rectangle	A weak entity, identified by its owner's key together with its partial key.
Single / double diamond	A binary relationship / an identifying relationship for a weak entity.
Oval / branched oval	An attribute / a composite attribute with its parts attached.
Double oval / dashed oval	Several values are allowed / the value is calculated.
Solid underline	The chosen key. Any other unique attributes are stated in the rules.
Dashed underline	The partial key of a weak entity. BookingDate and StartTime together are Booking's partial key.
Single / double connection	Participation is optional / participation is required.
(min,max) beside an entity	The minimum and maximum number of times one entity instance takes part. N means many.

How to read a constraint

Read the bounds from the entity beside them. A Season with (0,10) can have zero to ten episodes; an Episode with (1,1) must belong to exactly one season. The Episode side uses a double line because participation is required.

Additional rules

A diamond always joins two entity roles in these drawings. Adding an attribute to the diamond does not make it a third role. We write the extra checks beside each use case: for example, date restrictions, subset conditions and totals. EmployeeCount* is calculated automatically but is also stored.

Use Case 1

Who Wants to Be a Millionaire?

The show has two sets of records: seasons and episodes for production, and questions, attempts and joker uses for the contestants' games. The two views show these parts of the same model.

Entity identities

Entity	Chosen full key
Season	SeasonID
Episode	EpisodeID
Host	HostID
Contestant	ContestantID
Question	QuestionID
Category	CategoryID
QuestionAttempt (weak)	(ContestantID, AttemptNo)
JokerUse (weak)	(ContestantID, UseNo)
JokerType	TypeName

Design decisions

We use QuestionAttempt to keep what happened when a question was asked. It belongs to a Contestant, and AttemptNo is its partial key. This number only has to be unique within that contestant's game; it also gives the question order. UsesQuestion points to the question from the question bank.

A joker use needs its own record because it has a type, a time and an attempt it was used for. JokerUse is weak under Contestant, with UsedFor pointing to the attempt and OfType pointing to JokerType. This gives us binary relationships throughout. The maximum of four uses is a separate condition: it comes from the (0,4) at Contestant in HasJokerUse.

ChoiceA, ChoiceB, ChoiceC and ChoiceD hold the four answers. CorrectChoice gives the label of the correct answer. A question can belong to several categories, but it must belong to at least one.

For both Host and Contestant, FullName is made up of FirstName and Surname. Contestant.Age is derived from DateOfBirth and counts completed years.

Use Case 1 - Rules and assumptions

Business rules and additional constraints

1. Ten is the maximum number of episodes in a season. An episode belongs to one season and has one host. A person can enter the show only once, so we keep one Contestant record for that person and link it to a single episode.

2. HostID is the chosen host key, and Host.Email must also be unique. We keep EpisodeNumber as an ordinary attribute because the brief gives no global uniqueness requirement for it.

3. For a question, we store the question text and the four answer slots A, B, C and D. CorrectChoice must be one of those labels. DifficultyLevel holds values such as easy, medium or hard. A question needs at least one category, although it may have several.

4. A game cannot finish without at least one QuestionAttempt. Within the game, AttemptNo is unique and increases in the order the questions are asked. GivenAnswer is the submitted answer label. We store IsCorrect and require it to agree with GivenAnswer = Question.CorrectChoice.

5. Jokers are optional, with at most four JokerUse records per contestant. UsedAt records the date and time of a use. Every use has one joker type and refers to one QuestionAttempt owned by that same contestant.

6. The available joker types are Fifty-Fifty, Phone a Friend and Ask the Audience. JokerType keeps a description of each.

Assumptions

A1. We allow the question bank and production records to be prepared in advance. Seasons, planned episodes, hosts, categories and questions can therefore exist before being used. For participation limits that the brief leaves unspecified, we chose a minimum of zero.

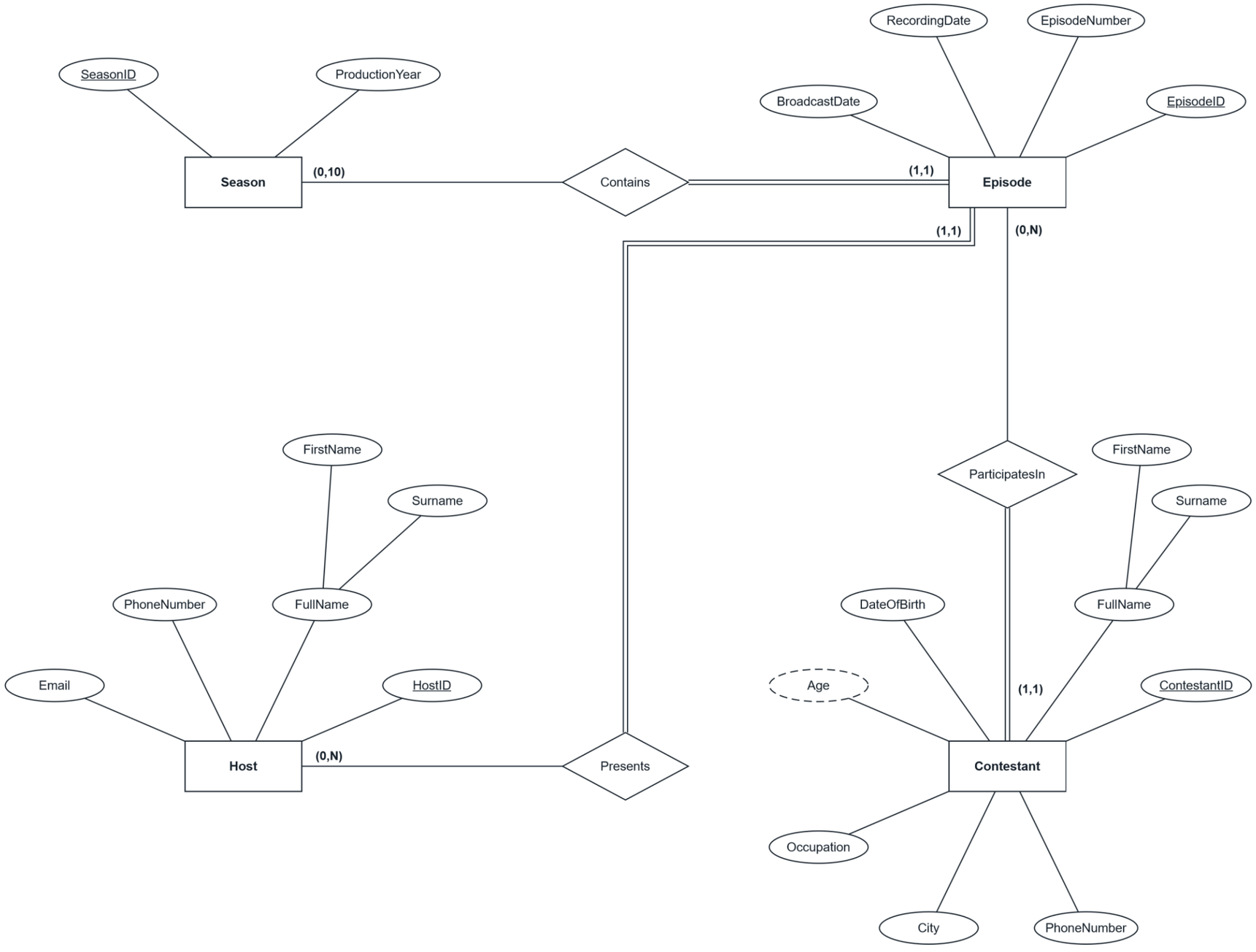
A2. The same question may be asked to the same contestant more than once. Each occurrence has a different AttemptNo. The brief does not rule out repeated questions.

A3. All joker uses count towards the same limit of four. A contestant may choose the same type again within that limit, since the brief does not set a separate limit for a type.

A4. To have a Contestant record, a person must already have been asked a question. This is why the Contestant side of HasAttempt has total participation.

Use Case 1A - Production and participation

Chen ERD | Entity attributes continue in View 1B where marked. All relationships are binary.



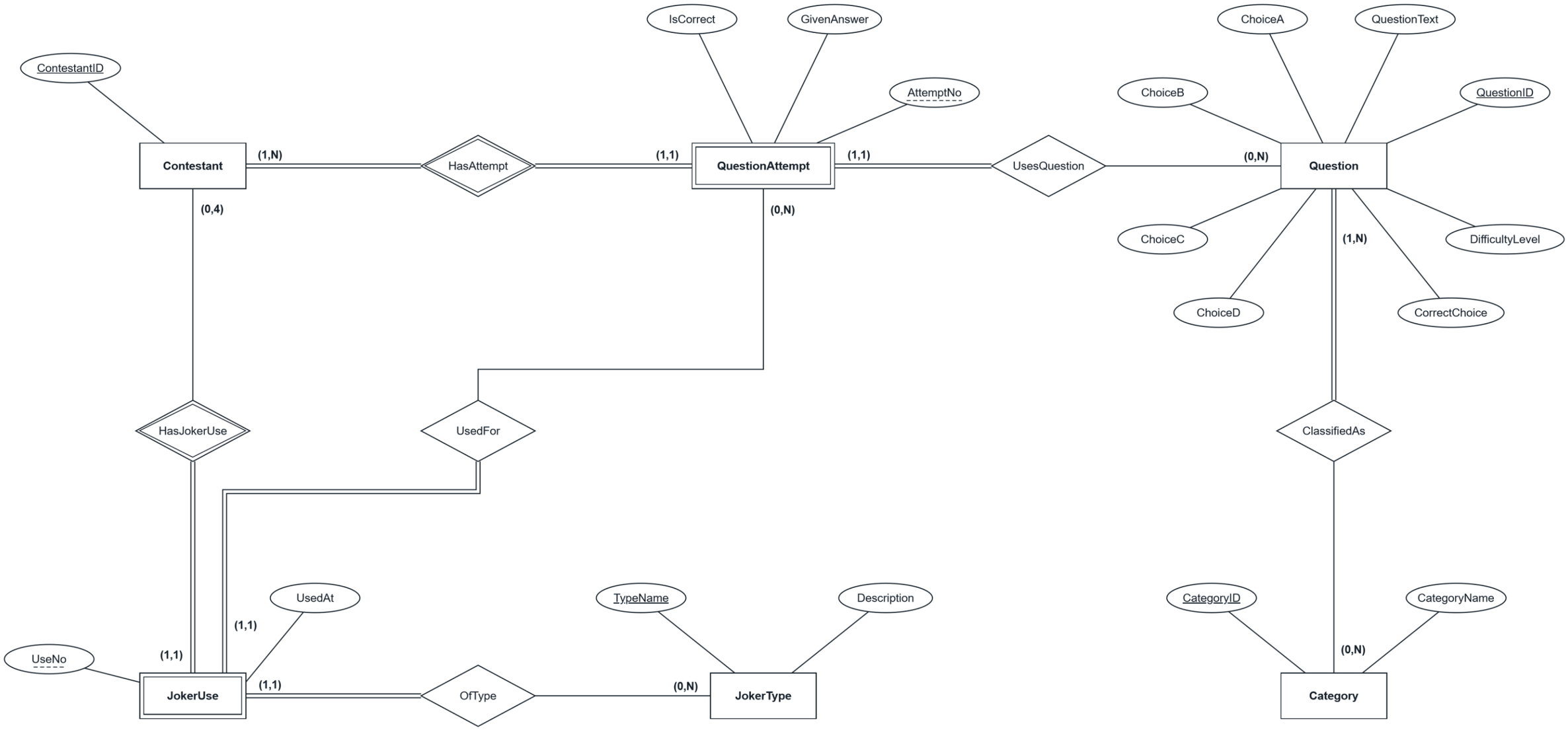
Notes

Contestant.Age is the number of completed years since DateOfBirth. Host.Email is unique in addition to HostID.

One Contestant record is kept for each person. It belongs to one episode, so a person can take part in the show only once.

Use Case 1B - Questions, answers and jokers

Chen ERD | Contestant is the same entity as in View 1A. Dashed underlines denote partial keys.



Notes

AttemptNo follows the question order within a game. The full keys are (ContestantID, AttemptNo) for QuestionAttempt and (ContestantID, UseNo) for JokerUse. UsedFor links a joker use to an attempt by the same contestant. We keep IsCorrect as a stored value and check it against GivenAnswer = Question.CorrectChoice. JokerType.TypeName is one of the three listed options: Fifty-Fifty, Phone a Friend or Ask the Audience.

Use Case 2

CleanCampus

Several dormitories can share a laundry machine. CleanCampus keeps the allocation details for those pairs, as well as student residence records, usage sessions and maintenance agreements.

Entity identities

Entity	Chosen full key
LaundryRoom	RoomID
Machine	MachineID
MachineModel	ModelID
Dormitory	DormID
Student	StudentID
Stay	StayID
UsageSession (weak)	(MachineID, SessionNo)
Supplier	SupplierID
MaintenanceContract	ContractID

Design decisions

A change to a model's specifications should apply to all machines of that model. We therefore put Capacity and SupportedPrograms on MachineModel. Capacity has two parts, MinLoad and MaxLoad, while SupportedPrograms can contain several names. Each machine has one model. Brand stays on Machine, as required by the brief.

UsageRank and FreeCreditAmount depend on which dormitory is using which machine. They belong to AllocatedTo, the binary many-to-many relationship between Machine and Dormitory. IsLeadingDorm is calculated from UsageRank for that particular pair.

We give each period of residence a StayID and link it to one Student and one Dormitory. A student who returns to a dormitory gets a new Stay record. CurrentlyIn shows the student's current dormitory and must agree with the open Stay.

SessionNo is only unique within one machine, so UsageSession is weak under Machine and the connection is identifying. A maintenance agreement works differently: MaintenanceContract has a ContractID of its own. A renewal can then be recorded separately even when it involves the same supplier and machine.

Use Case 2 - Rules and assumptions

Business rules and additional constraints

1. Machine.Type is standard or industrial. Dormitory.Level is undergraduate or graduate. Student.Username is unique in addition to StudentID. Student.Age is stored because the brief gives no birth date.

2. A machine has one MachineModel and one LaundryRoom, both mandatory. The model lists its supported program names. Its load bounds must satisfy MinLoad <= MaxLoad.

3. We keep no more than one allocation for a Machine-Dormitory pair. Its UsageRank and FreeCreditAmount are specific to that pair. IsLeadingDorm is true exactly when UsageRank = 1 for that machine.

4. CurrentlyIn gives each student one current dormitory. Stay keeps the residence periods, including at least one record for every student. There must be exactly one open Stay, with EndDate = NULL, and its dormitory must match CurrentlyIn. A return to the same dormitory gets a new StayID. Periods belonging to the same student cannot overlap.

5. Two machines can use the same SessionNo; the number is unique within a machine. The four Status values are scheduled, completed, cancelled and interrupted. Duration is derived from StartTime and EndTime once both are known.

6. For a completed or interrupted session, the actual-use dormitory is required. The machine must be allocated to it at the time of use. Scheduled and cancelled sessions may have no actual-use dormitory.

7. A maintenance contract connects one supplier with one machine and records StartDate, EndDate and ContractAmount. Renewing an agreement creates a separate contract record, including when the supplier-machine pair stays the same.

Assumptions

A1. RoomID identifies the shared laundry room where a machine is installed. We record one room per machine at a time and do not keep earlier room assignments.

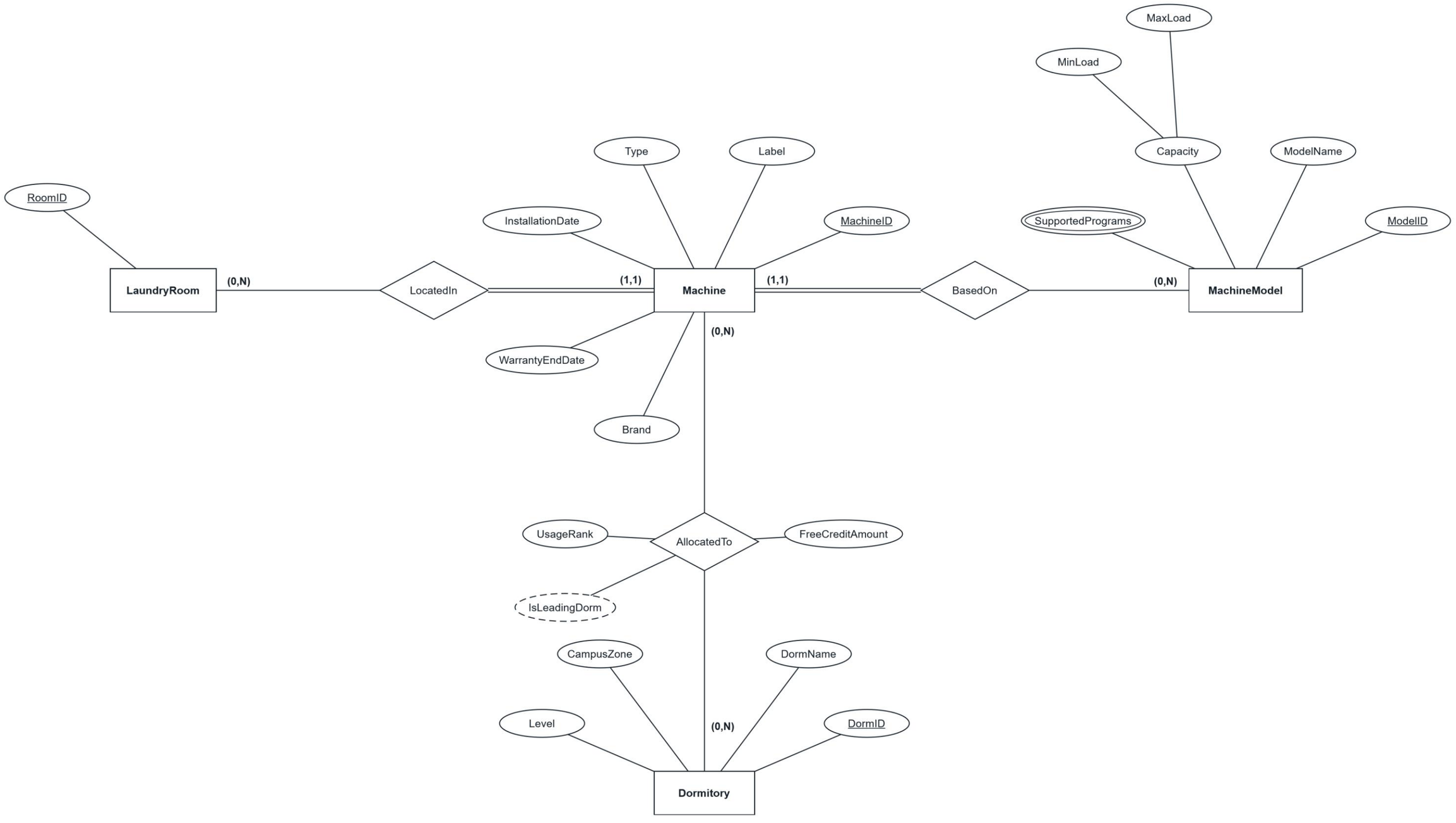
A2. The current open Stay is part of the stay history. On leaving a dormitory, the student's EndDate is filled in, and CurrentlyIn is kept consistent with the Stay record.

A3. For one machine, UsageRank values are distinct positive integers. Rank 1 belongs to its single leading dormitory.

A4. We treat a session as ending on its SessionDate. An unfinished or cancelled session may have an unknown EndTime. Duration is calculated only when StartTime and EndTime are both available.

Use Case 2A - Equipment and dormitory allocation

Chen ERD | Allocation attributes belong to the Machine-Dormitory pair.



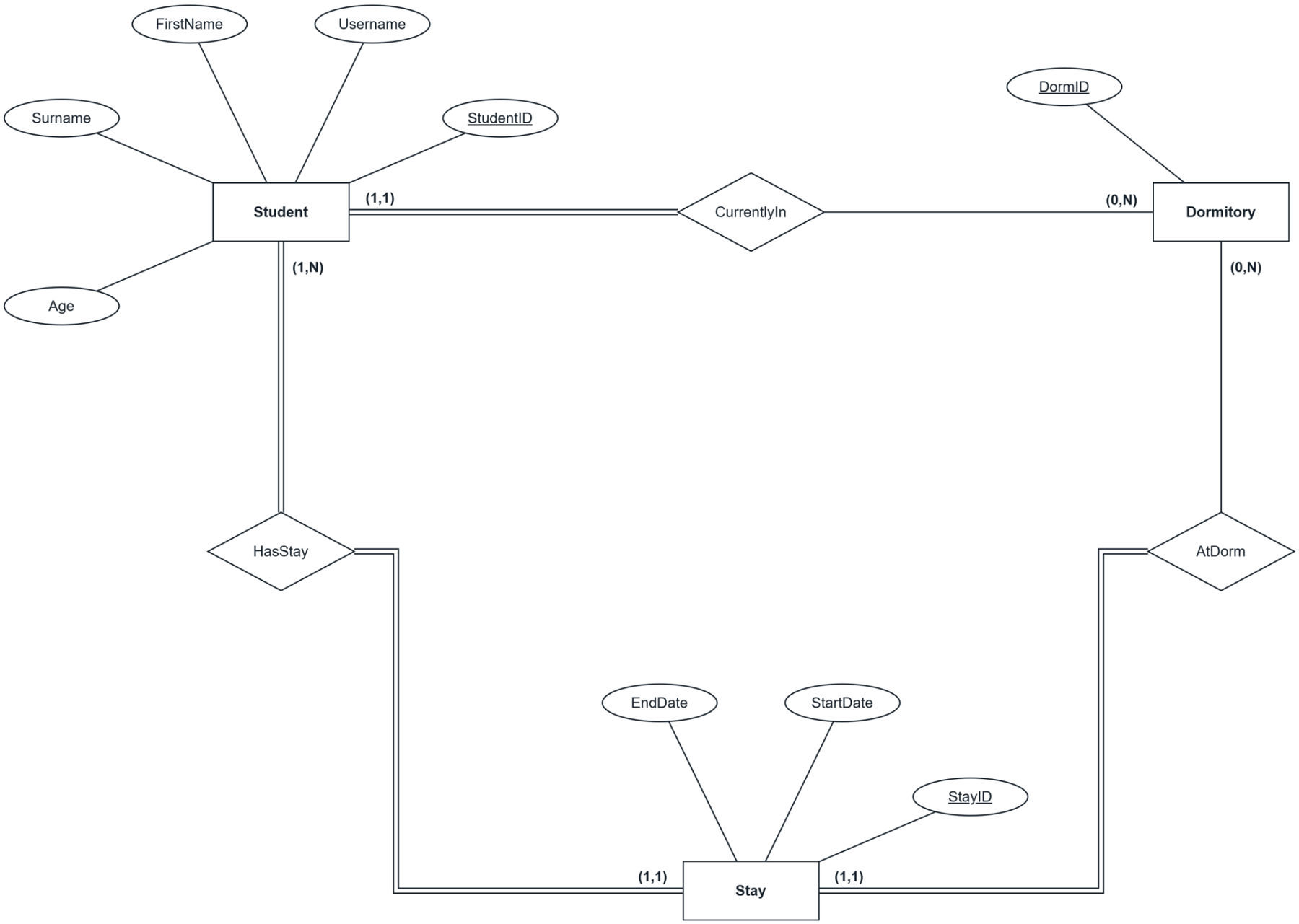
Notes

IsLeadingDorm is derived separately for each machine and is true exactly when UsageRank = 1.

Capacity is composite, with MinLoad and MaxLoad as its components. SupportedPrograms is multivalued.

Use Case 2B - Current residence and stay history

Chen ERD | Dormitory is the same entity as in View 2A. Each separate stay has its own StayID.



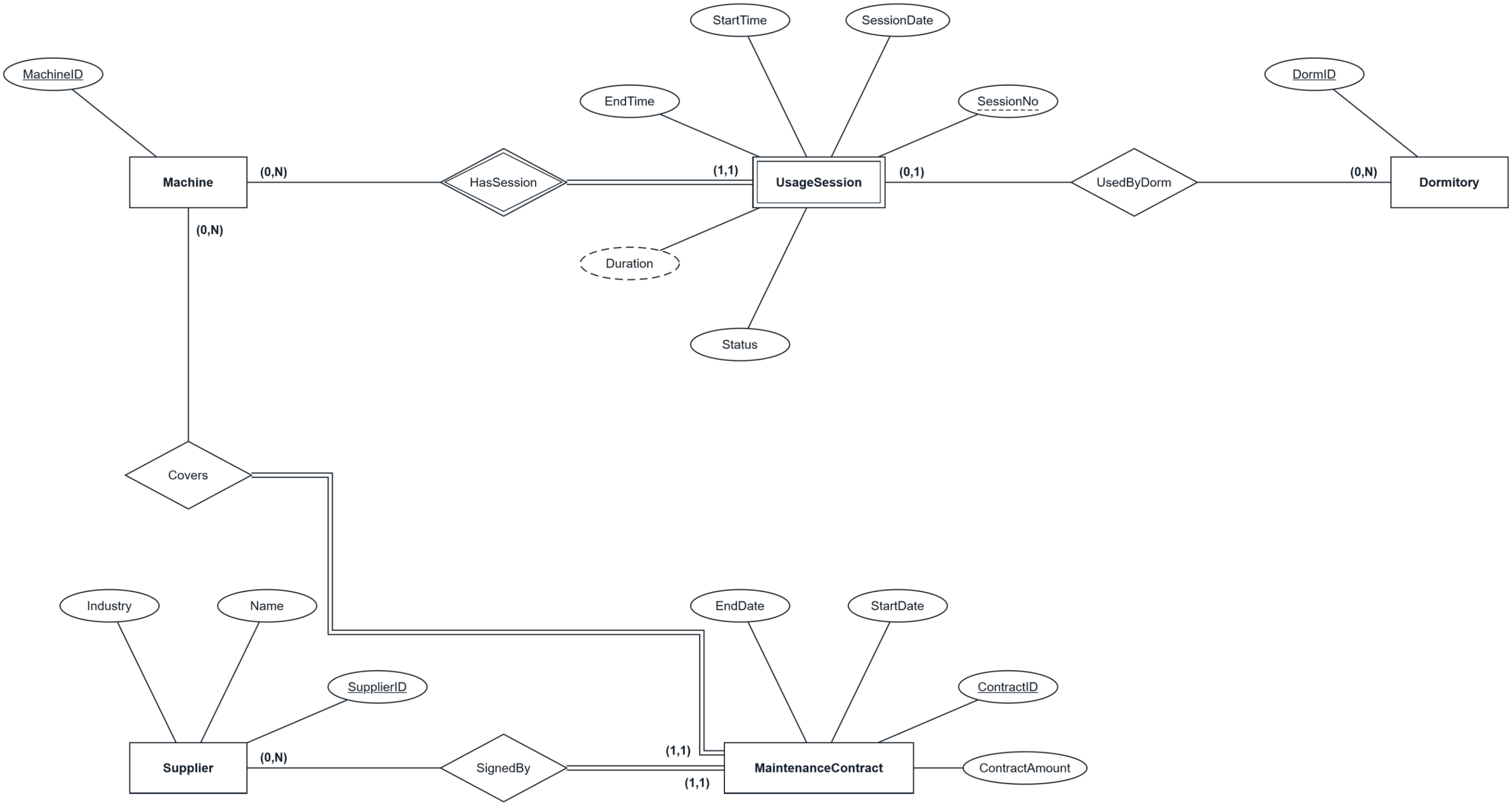
Notes

Student.Username is unique on its own, in addition to StudentID. Student.Age is stored because no birth date is given.

Each student has one open Stay with EndDate = NULL, and its dormitory must match CurrentlyIn. A return to the same dormitory gets a new StayID.

Use Case 2C - Usage sessions and maintenance

Chen ERD | Machine and Dormitory are references to View 2A. A usage session is weak under its machine.



Notes

UsageSession has the full key (MachineID, SessionNo). Duration is derived from StartTime and EndTime once both are available.
Completed and interrupted sessions must identify a dormitory. The machine must be allocated to that dormitory at the time of use.
A new maintenance agreement gets its own ContractID, even when the supplier and machine are the same.

Use Case 3

METU NCC KALTEV

The KALTEV model includes companies, their current staff and projects, office rentals, internships, meeting-room bookings and investments. Rentals and investments have separate records so that earlier transactions are retained.

Entity identities

Entity	Chosen full key
Company	TaxNumber
Employee	NationalID
Office	OfficeNumber
Rental	RentalID
Project	ProjectCode
Deliverable (weak)	(ProjectCode, DeliverableNo)
Intern	StudentID
MeetingRoom	RoomNumber
Booking (weak)	(RoomNumber, BookingDate, StartTime)
Investor	InvestorID
Investment	InvestmentID

Design decisions

A company can rent an office again or receive more than one investment. Rental and Investment therefore have their own identifiers. Each Investment is a dated transaction; these records are used to calculate Company.TotalInvestmentReceived and totals for an investor-company pair.

DeliverableNo is local to a Project. For Booking, the partial key (BookingDate, StartTime) is local to a MeetingRoom. Both weak entities have identifying relationships to their owners.

Company.Contact consists of Email and PhoneNumber. Employee.PhoneNumbers can hold several values, and CompanyAge is derived from FoundingDate. EmployeeCount* is stored even though it is calculated: it counts current WorksFor links and is updated automatically when those links change. The asterisk marks this storage requirement.

Use Case 3 - Rules and assumptions

Business rules and additional constraints

1. Company is identified by TaxNumber, and Employee by NationalID. Employee.Email must also be unique. Each employee currently works for one company through WorksFor, which records EmploymentStartDate.

2. Each company has one representative. An employee may represent at most one company and must currently work for that company. Represents is a subset of WorksFor for the same employee-company pair. This also means every company has at least one employee.

3. Each Rental links one company to one office and records StartDate, EndDate and MonthlyRent. A rental is ongoing when StartDate is on or before today and EndDate is NULL or later than today. Every company currently operating in KALTEV has at least one ongoing rental. Ended rentals are kept as history; an office may have no ongoing rental. Rental periods for the same office must not overlap.

4. A project belongs to one company. Its Status is proposed, active, completed or cancelled, and it has at least one Deliverable. Deliverable numbering starts at D1 for each project. ActualSubmissionDate may be NULL until the deliverable is submitted.

5. WorksOn records HoursPerWeek for each employee-project pair. Every project has one manager who also works on it, so Manages is a subset of WorksOn.

6. An intern belongs to one company through InternsAt and has one supervisor. The supervisor must work for the same company through WorksFor. Only current internships are recorded; the Intern record is removed when the internship ends.

7. There are exactly ten MeetingRoom records. Each booking is for one room and one company. Its full key is (RoomNumber, BookingDate, StartTime), so two bookings for the same room cannot start at the same time on the same date. Bookings in different rooms can start together.

8. An Investment has one investor and one receiving company. Company.TotalInvestmentReceived is the sum of Amount for that company's investments, or zero if it has none. To find the total from investor i to company c, we sum the investments linked to both i and c.

Assumptions

A1. WorksFor records current employment only. Previous employers are outside the model's scope. Employees work only on projects run by their current company.

A2. Rental periods are half-open intervals [StartDate, EndDate). A NULL EndDate means the interval is open-ended. A company may rent the same office again with a new RentalID.

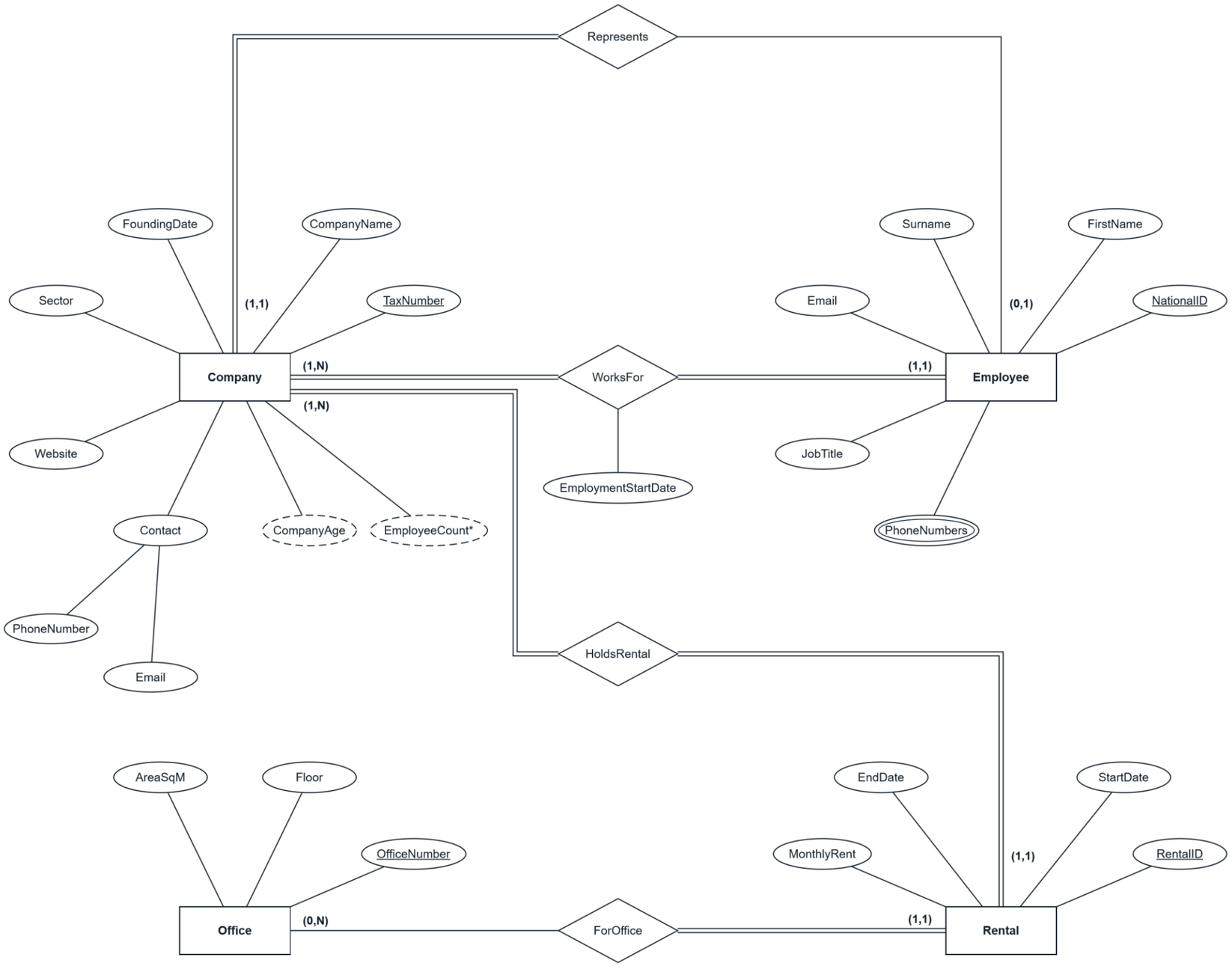
A3. A booking starts and ends on the same date, with StartTime earlier than EndTime. The key (RoomNumber, BookingDate, StartTime) prevents two bookings for the same room from starting on the same date at the same time.

A4. An investor may invest in a company more than once; each dated transaction has a separate InvestmentID. Investor email is not assumed to be unique. An investor may be registered before making an investment, which gives Investor partial participation in ProvidedBy.

A5. A company stays in the system after leaving KALTEV, with its past rentals still linked to it. Only companies currently operating in the building are required to have an ongoing rental.

Use Case 3A - Companies, employees and offices

Chen ERD | Company.EmployeeCount* is a stored, automatically maintained derived value.



Notes

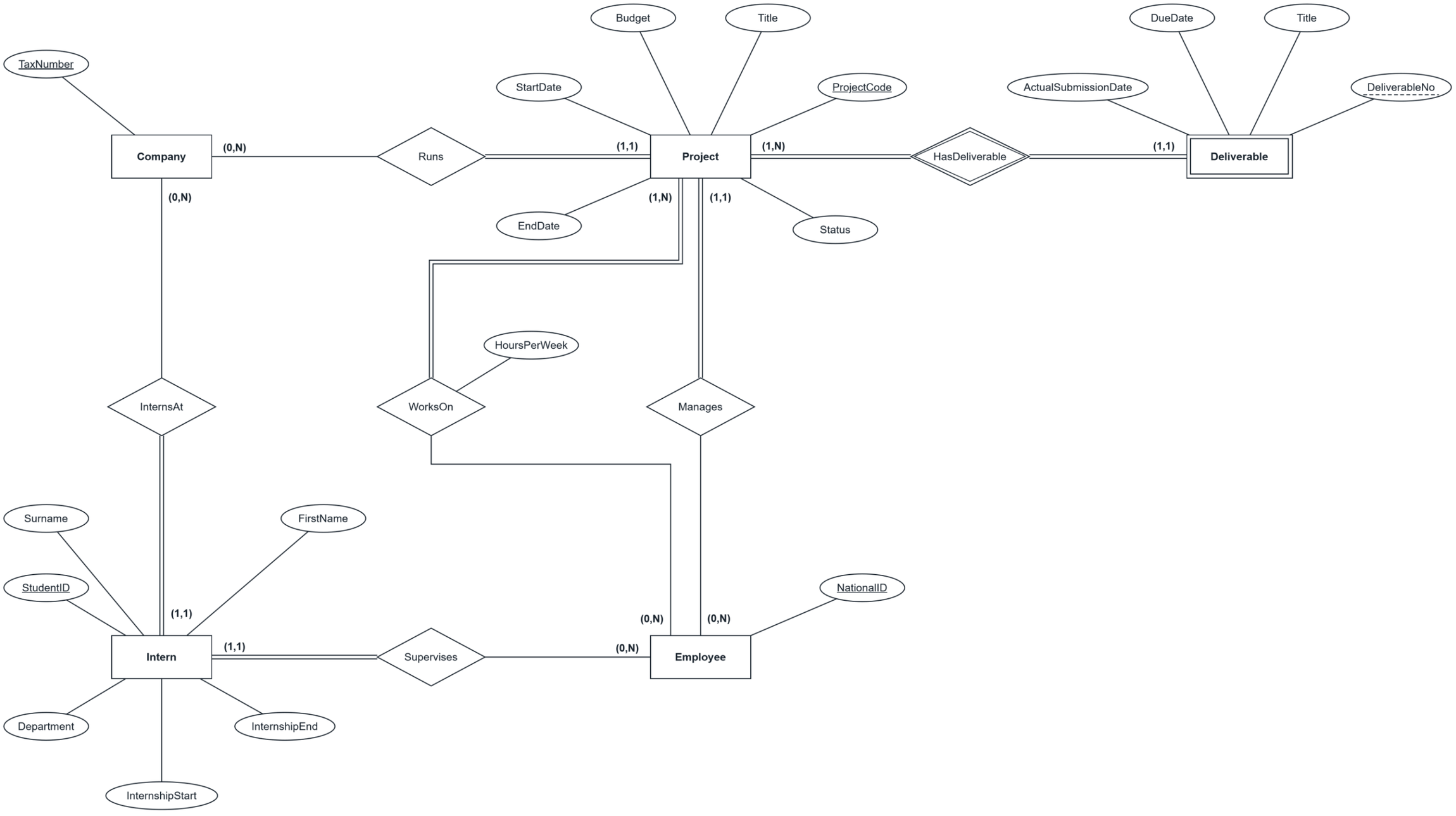
A company's representative must be one of its own current employees. Represents is therefore a subset of WorksFor.

Every company currently operating in KALTEV needs at least one ongoing rental. Rental periods for the same office must not overlap.

Employee.Email is unique in addition to NationalID. EmployeeCount* is the stored count of current employees in WorksFor and is updated automatically.

Use Case 3B - Projects, deliverables and interns

Chen ERD | Company and Employee refer to View 3A. Every project manager must also work on that project.

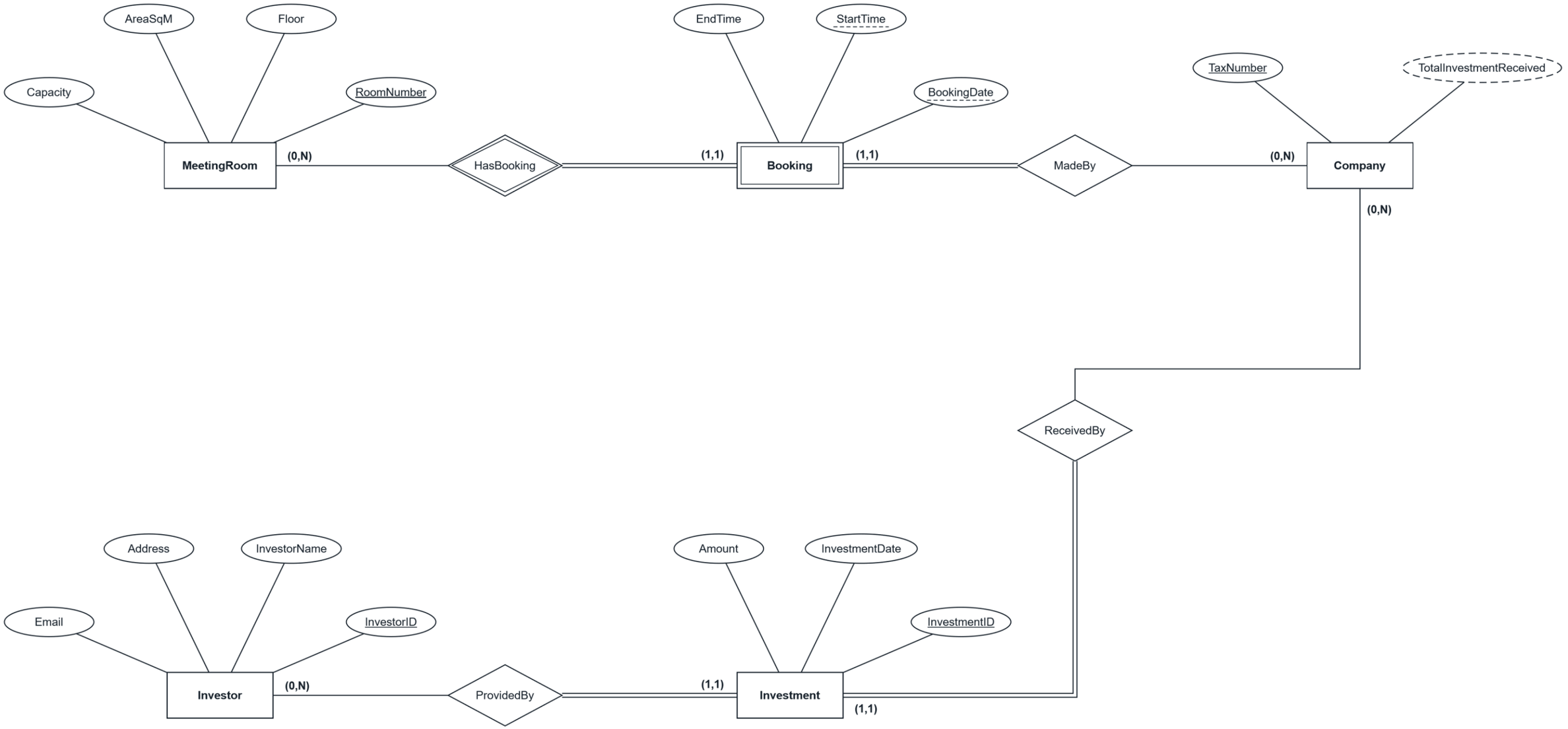


Notes

The full key for Deliverable is (ProjectCode, DeliverableNo). Numbering starts at D1 again for each project.
Manages is a subset of WorksOn: a project manager must also work on that project. All project workers belong to the company running it.
An intern and their supervisor must work for the same company. Only current internships are kept.

Use Case 3C - Meeting rooms and investment

Chen ERD | Company is the same entity as in View 3A; TotalInvestmentReceived is shown here.



Notes

Booking is identified by (RoomNumber, BookingDate, StartTime). Its partial key is (BookingDate, StartTime).

There are exactly ten MeetingRoom records. Each booking links one room to one company.

Each investment is kept as a separate transaction. Totals sum Amount for either a company or an investor-company pair.